
Fuzzing

Secure Software & Hardware Systems

Today

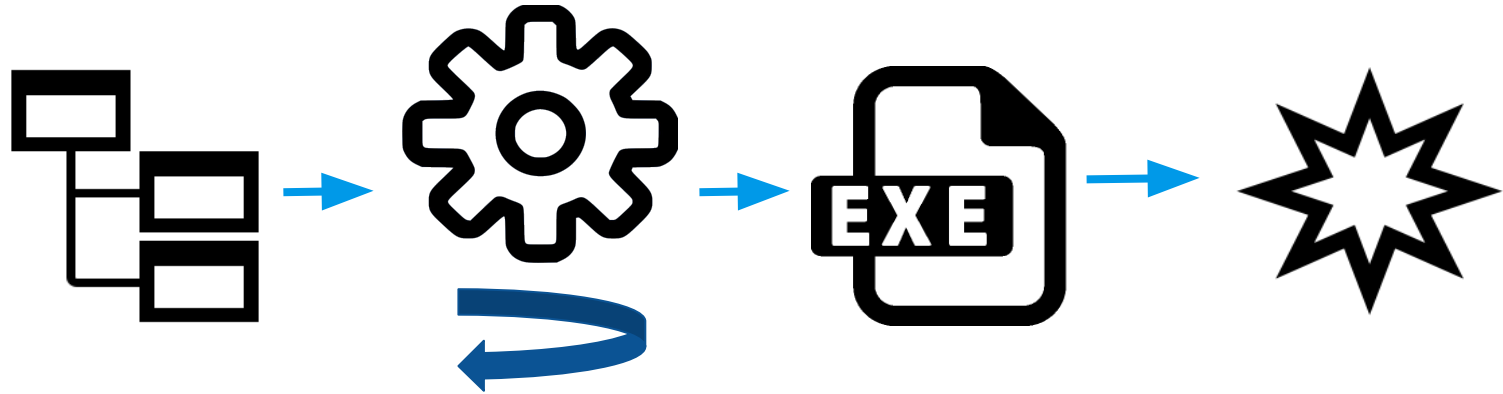
- Introduction
- Modern Fuzzing
 - Taxonomy & Terminology
 - Core algorithm
- AFL/AFL++ deep dive
 - History & inner workings
 - Advanced topics
- Fuzzing beyond AFL/AFL++
 - Improving fuzzing efficacy
 - Alternative fuzzer showcase
- Recap

Today

- Introduction
- Modern Fuzzing
 - Taxonomy & Terminology
 - Core algorithm
- AFL/AFL++ deep dive
 - History & inner workings
 - Advanced topics
- Fuzzing beyond AFL/AFL++
 - Improving fuzzing efficacy
 - Alternative fuzzer showcase
- Recap

Fuzzing?

Fuzzing delivers a large amount of machine-generated inputs as quickly as possible to the target in order to find unexpected side-conditions.



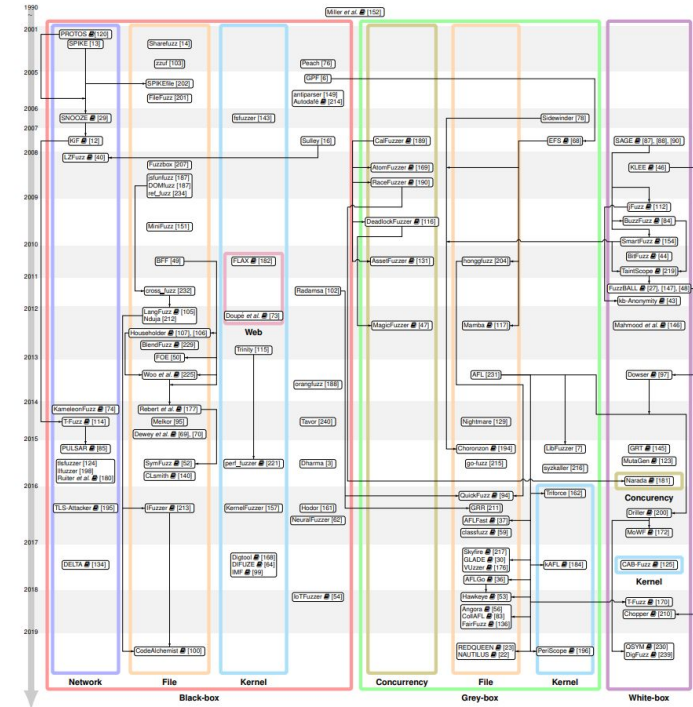
Fuzzing: Origin

1988: Class project introduced by Bart Miller at University of Wisconsin-Madison:

- Goal: test reliability of UNIX command line programs
- Idea: Feeding random inputs to programs and check for crashes
- First time usage of the term “fuzz” to describe testing approach
- Crashed up to 25-33% of programs(!)
- Results subsequently published in 1990

Fuzzing: 30 years later

- One of the core automated security testing techniques!
 - Incredible amount of fuzzers
 - Wide adoption in the industry
 - Large body of academic work
- Core principles of classic fuzzing still in-tact, however:
 - Smarter ways of input construction
 - Potentially more knowledge about the target



Today

- Introduction
- Modern Fuzzing
 - Taxonomy & Terminology
 - Core algorithm
- AFL/AFL++ deep dive
 - History & inner workings
 - Advanced topics
- Fuzzing beyond AFL/AFL++
 - Improving fuzzing efficacy
 - Alternative fuzzer showcase
- Recap

Fuzzing: Taxonomy

Modern fuzzers are usually described as triplet:

- Amount of feedback
 - Traditional Blackbox, Greybox, or Whitebox
 - Typical feedback: code coverage
- Input creation strategy
 - Mutation-based vs Generation-based
- Target
 - Source vs. Binary

Feedback

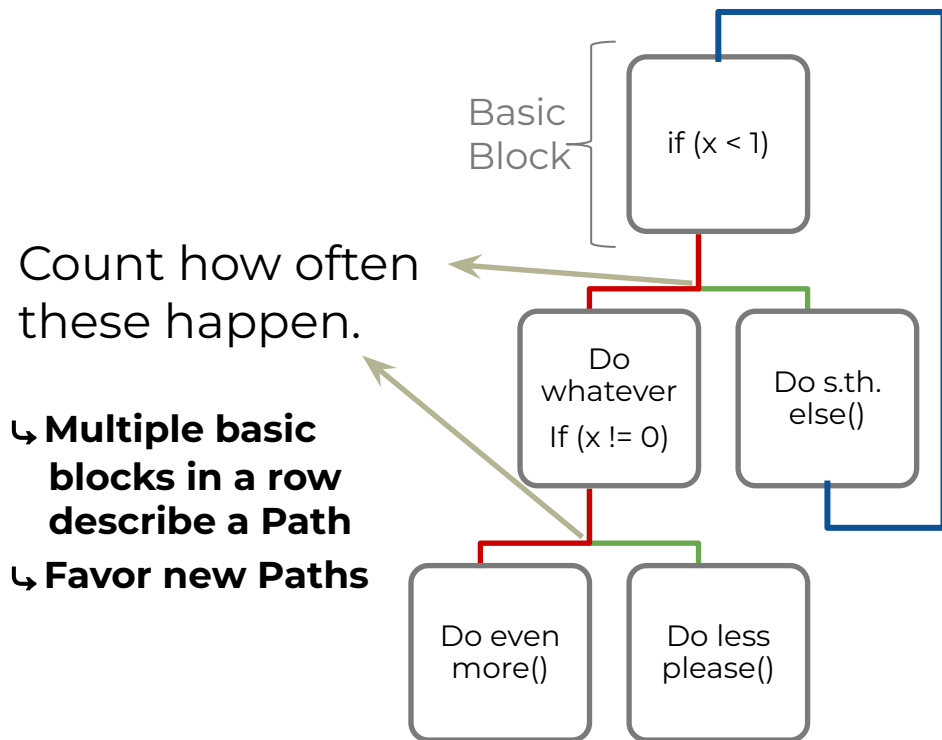
Traditional classification:

- Blackbox: No insight in the target whatsoever
 - Greybox: Retrieving some sort of feedback from the target
 - Focus on light-weight approaches
 - Whitebox: Insight over full program state
-
- Potentially better classification:
 - Feedback-driven vs. no feedback

Feedback-driven Fuzzing

- Core idea of modern fuzzing is using *feedback* from individual fuzzing runs to guide test case creation
- Feedback examples:
 - Code coverage
 - Response times
 - State coverage
- Requires instrumentation of target or execution environment
 - May induce performance penalty over plain black-box fuzzing
 - May allow to explore target faster → acceptable trade-off

Feedback Example: Coverage-Guided Fuzzing



- Most common form of feedback-driven fuzzing
- Code coverage as feedback
- Fuzzer keeps track of reached path and favors new paths

Examples:

AFLplusplus, LibAFL, Honggfuzz,
LLVM libFuzzer, Jackalope,
Syzkaller

Instrumentation?

“Instrumenting” a program means to change its functionality

- This requires the additions/modifications to the program's code

Typical ways of instrumentation:

- Source-based
- Compile-time based
- Binary-based

```
; int __fastcall main(int argc, const char **argv, const char **envp)
public main
main proc near

var_A0= qword ptr -0A0h
var_94= dword ptr -94h
s= byte ptr -90h
var_8= qword ptr -8

; __unwind {
endbr64
push    rbp
mov     rbp, rsp
sub     rsp, 0A0h
mov     [rbp+var_94], edi
mov     [rbp+var_A0], rsi
mov     rax, fs:28h
mov     [rbp+var_8], rax
xor     eax, eax
```



```
; int __fastcall main(int argc, const char **argv, const char **envp)
public main
main proc near

var_D8= qword ptr -0D8h
var_D0= qword ptr -0D0h
line= byte ptr -0C8h
var_40= qword ptr -40h

argc = rdi                ; int
argv = rsi                ; char **
; __unwind {
endbr64
push    r15
push    r14
push    r13
push    r12
push    rbp
push    rbx
sub     rsp, 0A8h
lea     r13, __afl_area_ptr
mov     r15, cs:off_AFD8
mov     rax, fs:28h
mov     [rsp+0D8h+var_40], rax
xor     eax, eax
lea     r12, service
lea     rbx, [rsp+0D8h+line]
lea     rbp, auth
mov     r14, [r13+0]
```

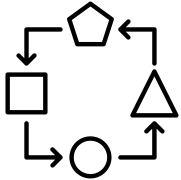
Input creation

Mutation-based

- Based on set of seed inputs
- Modifies existing input
- Especially effective when feedback is available

Generation-based

- Format of input is known beforehand
- Rules to derive inputs
- Often used when feedback not available



Fuzzing Target: Binary vs Source

Source-based

- Allows for efficient instrumentation and sanitization
- Usually: Feedback collection via compile time instrumentation

Binary-based

- Instrumentation & Sanitization is challenging
- Solutions for feedback collection:
 - Tracing via hardware features
 - Emulation
 - Binary Rewriting

0101
0011

Fuzzing: Additional Terminology

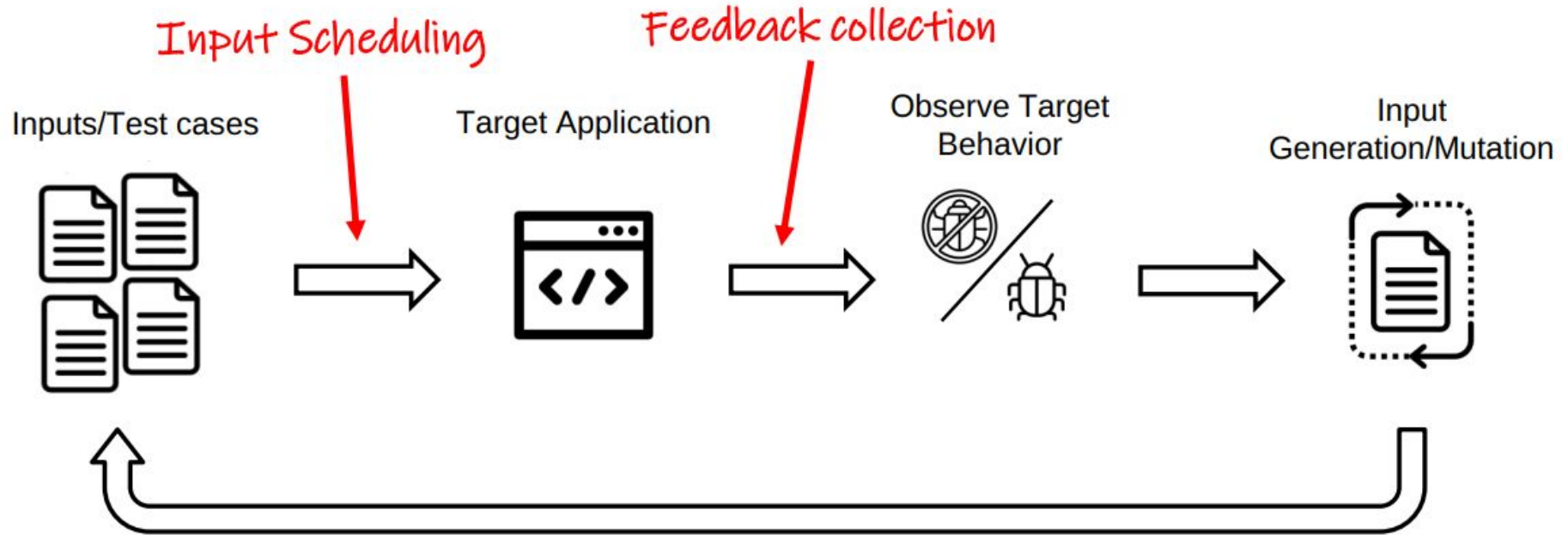
- Target: the software (or code unit) under test
- Harness: program to execute the target
- Corpus: A set of test inputs
- Seeds: The initial set of inputs given to the fuzzing process
- Queue: Test cases still to be executed
- Observation: Collected feedback of a test case execution



Today

- Introduction
- **Modern Fuzzing**
 - Taxonomy & Terminology
 - Core algorithm
- **AFL/AFL++ deep dive**
 - History & inner workings
 - Advanced topics
- **Fuzzing beyond AFL/AFL++**
 - Improving fuzzing efficacy
 - Alternative fuzzer showcase
- Recap

Fuzzing Algorithm



Fuzzing Algorithm

```
queue ← initSeedCorpus()
observations ← ∅
while ¬isDone(observations, queue) do
    candidate ← choose(queue, observations)
    mutated ← mutate(candidate, observations)
    observation ← eval(mutated)
    if isInteresting(observation, observations) then
        queue ← queue ∪ mutated
        observations ← observations ∪ observation
    end if
end while
```

Fuzzing algorithm (following definition of [1])

Fuzzing Algorithm

```
queue ← initSeedCorpus()
observations ← ∅
while ¬isDone(observations, queue) do
  candidate ← choose(queue, observations)
  mutated ← mutate(candidate, observations)
  observation ← eval(mutated)
  if isInteresting(observation, observations) then
    queue ← queue ∪ mutated
    observations ← observations ∪ observation
  end if
end while
```

Fuzzing algorithm (following definition by [1])

- **initSeedCorpus**: Initialize a new seed corpus.
- **isDone**: Determine if the fuzzing should stop or not based on progress toward a goal, or a timeout.
- **choose**: Choose at least one candidate seed from the queue for mutation.
- **mutate**: From at least one seed and any observations made about the program so far, produce a new candidate seed.
- **eval**: Evaluate a seed on the program to produce an observation.
- **isInteresting**: Determine if the observations produced from an evaluation on a mutated seed indicate that the input should be preserved or not.

Today

- Introduction
- Modern Fuzzing
 - Taxonomy & Terminology
 - Core algorithm
- **AFL/AFL++ deep dive**
 - History & inner workings
 - Advanced topics
- Fuzzing beyond AFL/AFL++
 - Improving fuzzing efficacy
 - Alternative fuzzer showcase
- Recap

american fuzzy lop ++4.08c {default} (python) [fast]

process timing

run time : 0 days, 0 hrs, 0 min, 27 sec

last new find : 0 days, 0 hrs, 0 min, 3 sec

last saved crash : none seen yet

last saved hang : none seen yet

cycle progress

now processing : 6.0 (17.6%)

runs timed out : 0 (0.00%)

stage progress

now trying : havoc

stage execs : 7240/12.8k (56.56%)

total execs : 7480

exec speed : 270.2/sec

fuzzing strategy yields

bit flips : disabled (default, enable with -D)

byte flips : disabled (default, enable with -D)

arithmetics : disabled (default, enable with -D)

known ints : disabled (default, enable with -D)

dictionary : n/a

havoc/splice : 0/0, 0/0

py/custom/rq : unused, unused, unused, unused

trim/eff : 0.00%/1, disabled

strategy: **explore**

state: **started :-)**

overall results

cycles done : 0

corpus count : 34

saved crashes : 0

saved hangs : 0

map coverage

map density : 0.19% / 0.53%

count coverage : 1.56 bits/tuple

findings in depth

favorable items : 9 (26.47%)

new edges on : 18 (52.94%)

total crashes : 0 (0 saved)

total tmouts : 0 (0 saved)

item geometry

levels : 2

pending : 0

pend fav : 0

own finds : 0

imported : 0

stability : 0

[cpu000]



AFL/AFL++

- Coverage-based, mutation-based fuzzer
 - Tremendous impact in industry and academia
- 3 main design principles:
 - Speed, reliability, ease of use [1]
 - Especially ease of use helped to success
- Software-suite rather than fuzzer alone
 - Tooling for instrumentation, human inspection, test minimization, and crash deduplication



History of AFL/AFL++

- 2013: Initial release of AFL
 - <https://lcamtuf.coredump.cx/afl/>
- 2017: Stop of active development for AFL (v2.52b)
 - <https://github.com/google/AFL>
- 2019: Start of community initiated AFL++ project
 - <https://github.com/AFLplusplus/AFLplusplus>
- 2020: Release of AFL++ paper
 - Fioraldi et al. AFL++: Combining Incremental Steps of Fuzzing Research. WOOT'20



AFL/AFL++: Core operation (simplified)

- afl-fuzz provides test case to target via stdin or files
 - Queue, crashes, hangs, and fuzzer stats are also all saved as files!
- Target executes testcase and tracks coverage via bitmap
 - Bitmap is analyzed by fuzzer after test case execution
 - If a test case generated new coverage, safes it in queue
- Input mutation via multiple different strategies
 - Repeatedly mutates and executes the same test file

The afl-fuzz Commandline

-i Where to read input seeds from
-o Where to place outputs

AFL_AUTORESUME=1	afl-fuzz	-t 1000	-M main	-m none	-i in -o out	-- ./vuln_program	-f @@
------------------	----------	---------	---------	---------	--------------	-------------------	-------

Environment variables control
AFL's behaviour.
Almost everything in AFL++ is
configurable by env variables.

Example:

AFL_AUTORESUME :
if restarted, resume fuzzing

the afl-fuzz command

timeout: 1 second per testcase

This is the main node (vs -S)

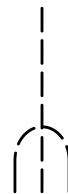
Memory limit

the target

@@:
Will be
replaced
with input
file (by
afl-fuzz)

AFL/AFL++ Forkserver

- Performance optimization
 - Fork program state after initialization (e.g., upon reaching `main()`)
 - `execve()`, linking, and libc initialization only executed once!
 - Parent (fuzzer) and child (target) communicate via pipes
 - Provides a ~2x performance gain
- Deferred mode
 - Allow the forkserver to spawn even later (e.g., after core configuration has been carried out)
 - Good for programs with lots of initial configuration
- Persistent mode
 - Fork only every n-th iteration
 - Good for programs with input-processing loops



AFL/AFL++ Instrumentation

- By default, added during compile time
 - AFL-CC as “drop-in” compiler replacement for clang/gcc:
 - `$ afl-cc target.c`
 - However, options for binary fuzzing exist as well, e.g., qemu-mode or frida mode
 - Adds coverage tracking and forkserver logic

```
cur_location = <COMPILE_TIME_RANDOM>;  
shared_mem[cur_location ^ prev_location]++;  
prev_location = cur_location >> 1;
```

AFL/AFL++ Mutation Strategies

Variety of strategies, for instance:

- Trim: Reduce test case to shortest form yielding same coverage
- Arith: subtraction or addition to input values
- Bitflip: flip certain bits in the input
- Havoc: combination of multiple strategies, including block deletion, duplication, and dictionary additions (if applicable)
- Sync: In multi-process fuzzing setups, testcases from other fuzzers



afl-showmap

- Tool to provide the results of coverage tracking for a given test case in human readable format
 - Allows also full directories as input
 - Useful environment variable: AFL_PRINT_FILENAMES
- Useful to replay a full fuzzing corpus against the target
 - Helps to identify potential roadblocks in fuzzing campaigns
 - Or detect differences between different crashes

Minimization tools

afl-cmin:

- Corpus Minimization
- Removes testcases that don't find new paths
- Smaller corpus $\Rightarrow$ focus on the important testcases

afl-tmin:

- Minimizes a single testcase
- Tries to find the smallest possible input

Today

- Introduction
- Modern Fuzzing
 - Feedback/Coverage-guided fuzzing
 - Terminology
 - Core algorithm
- **AFL/AFL++ deep dive**
 - Taxonomy & Terminology
 - Advanced topics
- Fuzzing beyond AFL/AFL++
 - Improving fuzzing efficacy
 - Alternative fuzzer showcase
- Recap

AFL/AFL++ QEMU Mode

- One example to allow binary fuzzing with AFL
 - Traditionally: Linux user-space programs
 - ~2-5x slowdown compared to source-based fuzzing
 - Allows for target binary with different ISA!
- Coverage collection for translated blocks
 - Inlined in QEMUs Tiny Code Generator (TCG)
 - Caching of translated blocks in forkserver parent
- We will hear more about QEMU during the rehosting lectures



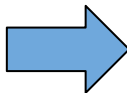
Multicore fuzzing

- Each instance of AFL/AFL++ binds to one CPU core
 - For multicore fuzzing, just spawn more instances!
 - Usually via tmux, screen, or some python script like afl-utils
- Important: use -M and -S flags:
 - Main node: -M main-\$HOSTNAME
 - Secondary nodes -S s1, -S s2, ...
- Common practice: diverse configs for different nodes
 - Test cases will be synchronized across nodes

COMPCOV / Compare Coverage

- Core idea: Split multi-byte compares to give feedback for single bytes.
 - This way, the fuzzer can make its way through the complete compare more easily
 - For 32-bit compares: brute-forcing 4×2^8 instead of 2^{32}

```
if (x == 0xdeadbeef) {  
    interesting_fn()  
}
```



```
if (x[0] == 0xde) {  
    if (x[1] == 0xad) {  
        if (x[2] == 0xbe) {  
            if (x[3] == 0xef) {  
                interesting_fn();  
            }  
        }  
    }  
}
```

CmpLog / Redqueen / WEIZZ

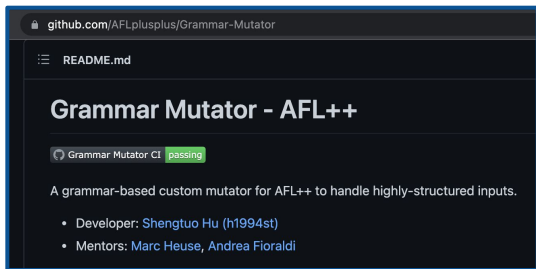
Simple Solving by tracking compares during runs.

```
user_input = "dog";  
strcmp(user_input, "cat");
```

⇒ If "dog" was in the input, the fuzzer can replace it with "cat"

- Slow but thorough
- Run on one of the cores in multi-core fuzzing

Grammar based fuzzing with AFL++



- AFL++ can use Grammars for better input generation
- You can also write custom mutators from scratch

```
"<A>": [{"<START_LINE>", "\r\n", "<HEADERS>", "<BODY>", "\r\n\r\n"}],

"<START_LINE>": [{"<METHOD>", " ", "<URI>", " ", "<VERSION>"}],

"<METHOD>": [{"GET"}, {"HEAD"}, {"POST"}, {"PUT"}, {"DELETE"}, {"CONNECT"}, {"OPTIONS"}, {"TRACE"}, {"PATCH"}, {"ACL"}, {"BASELINE-CONTROL"}, {"BIND"},

"<URI>": [{"<SCHEME> ", ":", "<HIER>", "<QUERY>", "<FRAGMENT>"}],

"<SCHEME>": [{"http"}, {"https"}, {"shhttp"}, {"dav"}, {"about"}, {"attachment"}, {"cid"}, {"data"}, {"file"}, {"ftp"}, {"ssh"}, {"sip"}],
```

Today

- Introduction
- Modern Fuzzing
 - Taxonomy & Terminology
 - Core algorithm
- AFL/AFL++ deep dive
 - History & inner workings
 - Advanced topics
- Fuzzing beyond AFL/AFL++
 - Improving fuzzing efficacy
 - Alternative fuzzer showcase
- Recap

Sanitizers

- Bugs, even when triggered, do not necessarily result in a crash
 - Sanitizers aim to change this
- Example sanitizers:
 - Address Sanitizers (ASan): Memory corruptions
 - ThreadSanitizer (TSan): Race conditions, deadlocks
 - Memory Sanitizer (MSAN): Uninitialized memory
 - UBSan: Undefined behaviour
- (Probably) most relevant for fuzzing: ASAN
 - For gcc/clang: added at compile time via `-fsanitize=address`
 - Supported by afl-cc via `AFL_USE_ASAN=1` environment variable

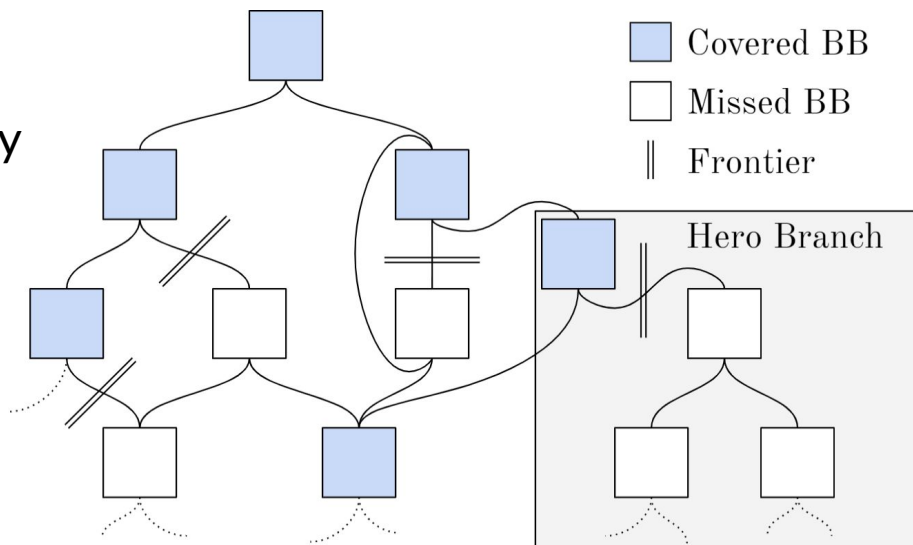
Example: Address Sanitizer Output

Fuzzing Roadblocks/Coverage Frontiers

Sometimes, the fuzzer gets stuck at a branch that hides a lot of functionality

Can be addressed by:

- Harnessing
- Seed selection
- Tooling



Cross-pollination

- Original term from botanics
 - One plant pollinates a plant of another variety
- Useful concept for fuzzing:
 - Using corpus from one target for another target
- Helpful when different libraries take same data format as input
 - For instance: jpg parsers

Fuzzing in CI/CD

- Some developers/companies moved over to use fuzzing as part of CI/CD
 - Allows to catch bugs without explicit test case generation
- Advice for fuzzing in CI/CD:
 - Always keep the corpus (with occasional minimization)
 - This will find more paths with each execution
- For AFL/AFL++:
 - Move `./out/default/queue` to `./in` for the next run,
 - Potentially run `afl-cmin` in-between

A note on sustainability

- Fuzzing may not be the most sustainable testing technique:
 - High CPU loads
 - Over multiple cores
 - Over long period of time
- Reasonable trade-off in terms of efficacy
 - Still, overall energy consumption should be considered carefully
 - And sources of energy
- No study about carbon footprint of fuzzing available*
 - Potential MSc Thesis topic

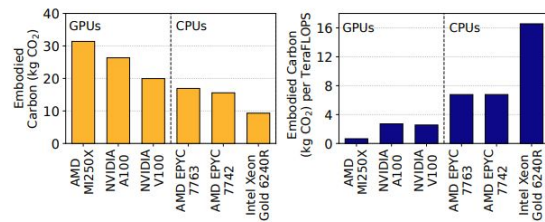


Figure 1: Embodied carbon footprint of GPU/CPU devices, and the footprint normalized to theoretical double-precision floating point performance (TeraFLOPS).

* To my knowledge

Today

- Introduction
- Modern Fuzzing
 - Feedback/Coverage-guided fuzzing
 - Taxonomy & Terminology
 - Core algorithm
- AFL/AFL++ deep dive
 - History & inner workings
 - Advanced topics
- Fuzzing beyond AFL/AFL++
 - Improving fuzzing efficacy
 - Alternative fuzzer showcase
- Recap

Fuzzer showcase: libfuzzer

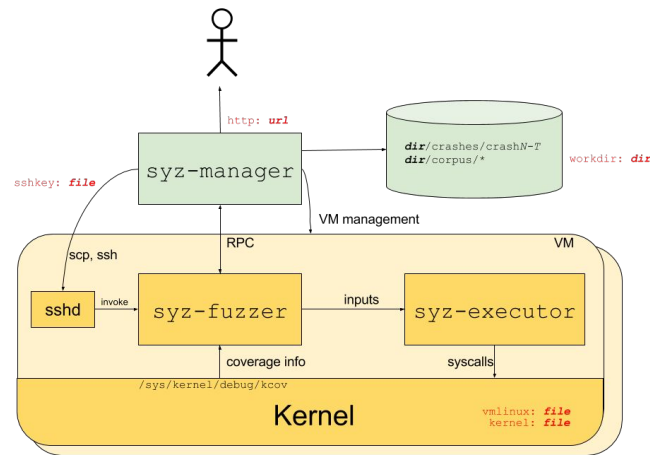
- Part of the LLVM infrastructure
- In-process, coverage-guided, mutation-based
- Focus in library/program testing from developers perspective
- Example:

```
// fuzz_target.cc
extern "C" int LLVMFuzzerTestOneInput(const uint8_t *Data, size_t Size) {
    DoSomethingInterestingWithMyAPI(Data, Size);
    return 0; // Values other than 0 and -1 are reserved for future use.
}
```

```
$ clang -g -O1 -fsanitize=fuzzer fuzz_target.cc
```

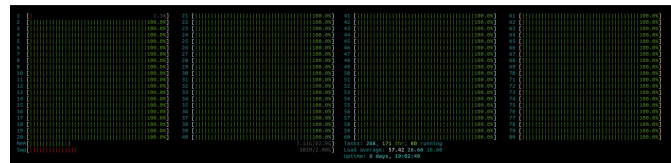
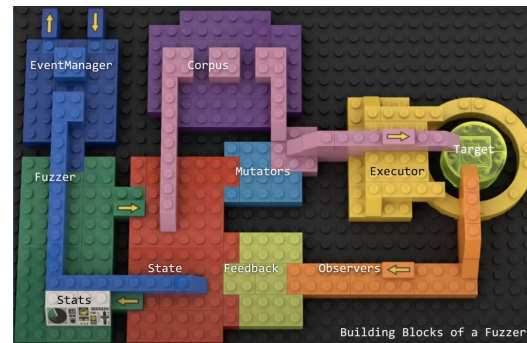
Fuzzer showcase: syzkaller

- Coverage-guided fuzzer dedicated to syscalls
 - Originally targeting Linux
 - Now also other operating systems
 - Uses a minimalistic syscall description syntax
- Couple of interesting challenges to solve:
 - Coverage collection (KCOV)
 - Unstable targets (VMs)
 - Input delivery (User space process)
 - Testcase synchronization (SSH)



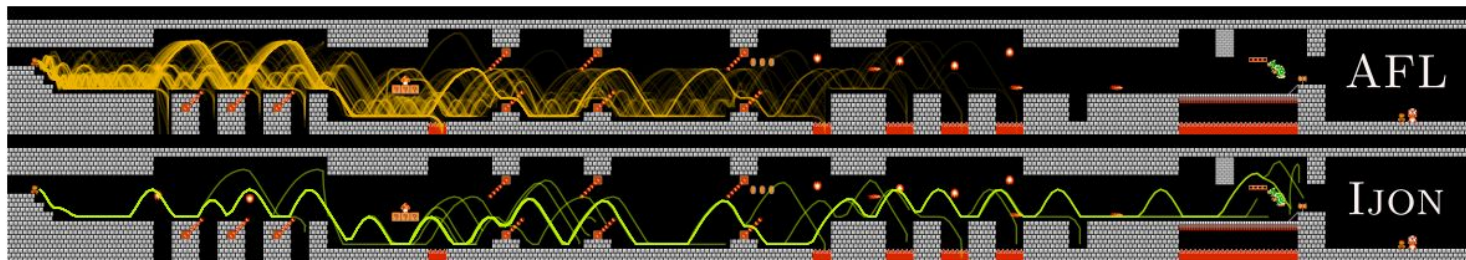
Fuzzer showcase: LibAFL

- Advanced Fuzzing Library
 - Developed by AFL++ maintainers
 - Not really related to AFL anymore
- Core idea:
 - Change how fuzzers are developed
 - Instead of stand-alone tools: building blocks
 - Provide library for easy integration & modification
- Resulting Impact:
 - Performance & scalability
 - Code Reusability/Portability (if you know Rust)
 - High flexibility



Fuzzer showcase: Honourable mentions

- Sandsifter: Fuzzing of x86 instructions!
 - <https://github.com/Battelle/sandsifter>
- Ijon: Allowing annotations for state variables
 - <https://ieeexplore.ieee.org/abstract/document/9152719>
- Nyx: Hypervisor Fuzzing with fast snapshotting
 - <https://www.usenix.org/system/files/sec21-schumilo.pdf>



Today

- Introduction
- Modern Fuzzing
 - Taxonomy & Terminology
 - Core algorithm
- AFL/AFL++ deep dive
 - History & inner workings
 - Advanced topics
- Fuzzing beyond AFL/AFL++
 - Improving fuzzing efficacy
 - Alternative fuzzer showcase
- Recap

Recap

- Introduction to modern fuzzing
 - What is the approach to state-of-the-art fuzzing?
- AFL/AFL++ DeepDive
 - How does a typical fuzzer work?
- Improving fuzzing efficacy
 - How can we make the fuzzing process more efficient?
- Fuzzer showcase
 - How do other successful fuzzers look like?

Additional resources & further reading

- Fuzzing book:
 - <https://www.fuzzingbook.org/>
- Libafl documentation:
 - <https://aflplus.plus/libafl-book/libafl.html>
- AFL++ technical details:
 - <https://blog.ritsec.club/posts/afl-under-hood/>
- Additional perspective on AFL's design choices:
 - Fioraldi et al. "Dissecting American Fuzzy Lop A FuzzBench Evaluation." FUZZING'22
- Recent paper on evaluating Fuzzing:
 - Schloegel et al. "SoK: Prudent Evaluation Practices for Fuzzing." IEEE S&P'24.